

AI 이메일 생성기 프로젝트 학습 노트

로컬 LLM부터 클라우드 배포까지 — 처음부터 끝까지

목차

1. AI와 LLM 기초
2. Llama 모델 이해하기
3. 모델 파일 포맷 — GGML과 GGUF
4. HuggingFace — AI 모델 저장소
5. CTransformers — 로컬에서 AI 실행
6. GPU vs CPU 속도 실험
7. Ollama — 더 쉬운 로컬 AI 실행
8. Streamlit — Python으로 웹 앱 만들기
9. LangChain — AI 연동 표준화
10. Groq — 무료 클라우드 AI API
11. 보안 — API 키 안전하게 관리하기
12. Git과 GitHub — 코드 버전 관리
13. Streamlit Cloud — 웹 배포
14. 전체 프로젝트 코드 해설
15. 핵심 개념 총정리

1. AI와 LLM 기초

AI 모델이란?

수많은 텍스트 데이터로 학습하여 사람처럼 글을 쓰고 이해하는 프로그램.

LLM(Large Language Model)이란?

대형 언어 모델의 약자. 수백억 개의 단어로 학습해 질문에 답하고, 글을 쓰고, 번역하는 등 언어 관련 작업을 수행함.

사용자: "회의 일정 안내 이메일 써줘"

LLM: "안녕하세요. 다음 주 화요일 오전 10시에 회의가 예정되어 있어..."

클라우드 LLM vs 로컬 LLM

구분	클라우드 LLM	로컬 LLM
예시	ChatGPT, Gemini, Claude	Llama, Mistral
실행 위치	회사 서버	내 컴퓨터

비용	사용량만큼 과금	무료
개인정보	서버로 전송됨	내 PC에서만 처리
인터넷	필요	불필요
성능	매우 좋음	하드웨어에 따라 다름

temperature(창의성 수치)란?

AI 답변의 다양성을 조절하는 값 (0.0 ~ 1.0)

```
temperature=0.0 # 항상 같은 답변 → 정확한 정보 전달에 적합
temperature=0.7 # 적당히 창의적 → 이메일, 문서 작성에 적합
temperature=1.0 # 매우 창의적, 다양함 → 소설, 시 창작에 적합
```

2. Llama 모델 이해하기

Llama란?

Meta(페이스북 모회사)가 무료로 공개한 오픈소스 LLM.
누구나 다운받아 상업적으로도 사용 가능.

모델 버전 비교

버전	출시	특징
Llama 2	2023년 7월	최초 오픈소스 공개. 7B/13B/70B
Llama 3	2024년 4월	성능 대폭 향상. 8K 컨텍스트
Llama 3.1	2024년 7월	128K 컨텍스트. 다국어 강화

파라미터(B) 숫자의 의미

모델 안에 있는 가중치(숫자) 개수.

```
7B = 70억 개 → 일반 PC에서 실행 가능 (RAM 8~16GB)
13B = 130억 개 → RAM 16~32GB 필요
70B = 700억 개 → RAM 64GB 이상 또는 고급 GPU 필요
```

숫자가 클수록 성능이 좋지만 더 많은 메모리가 필요함.
일반 개인 PC에서는 7B 또는 8B 모델이 현실적.

Chat 버전이란?

기본 모델은 "다음 단어 예측"만 함.
Chat 버전은 지시를 따르도록 추가 학습된 버전.

기본 모델: "안녕하세요 저는..." → "오늘 날씨가..." (그냥 이어 씀)
Chat 모델: "이메일 써줘" → "안녕하세요, 아래와 같이 이메일을..." (요청 수행)

3. 모델 파일 포맷 — GGML과 GGUF

왜 특별한 포맷이 필요한가?

일반 AI 모델 파일(.pt)은 GPU 없이 실행하기 매우 어려움.
CPU에서도 실행 가능한 경량 포맷이 별도로 개발됨.

GGML (구버전)

파일 확장자: .bin
개발자: Georgi Gerganov
특징: C/C++로 구현, CPU 최적화

이 프로젝트 파일: llama-2-7b-chat.ggmlv3.q8_0.bin (7.16GB)

파일명 읽는 법:

llama-2-7b-chat . ggmlv3 . q8_0 . bin
모델 이름 포맷 버전 양자화 확장자

GGUF (신버전, 현재 표준)

파일 확장자: .gguf
2023년 하반기부터 표준
메타데이터 포함, 더 안정적

양자화(Quantization)란?

모델 파일 크기를 줄이는 압축 기술.
원래 32비트 소수를 더 작은 비트로 표현함.

종류	비트 수	파일 크기	품질	추천 용도
Q8_0	8비트	~7GB	원본과 거의 동일	품질 중시
Q4_K_M	4비트	~4GB	약간 손실	균형 (추천)
Q2_K	2비트	~2GB	품질 낮음	저사양 PC

비유: 사진 압축과 같음
원본 사진(32비트) → JPG 80% 품질(Q4) → JPG 50% 품질(Q2)
파일은 작아지지만 화질(품질)이 조금씩 낮아짐

4. HuggingFace — AI 모델 저장소

HuggingFace란?

AI 모델과 데이터셋을 공유하는 플랫폼. "AI계의 GitHub".

- 웹사이트: <https://huggingface.co>
- 수십만 개의 무료 AI 모델 공유
- TheBloke 계정: 다양한 모델의 GGML/GGUF 버전을 변환해서 업로드하는 것으로 유명

모델 다운로드 방법

방법 1: 브라우저 직접 다운로드

```
https://huggingface.co/TheBloke/Llama-2-7B-Chat-GGML  
→ Files 탭 → 원하는 파일 클릭 → 다운로드
```

방법 2: Python 코드로 다운로드

```
pip install huggingface_hub  
  
from huggingface_hub import hf_hub_download  
  
hf_hub_download(  
    repo_id="TheBloke/Llama-2-7B-Chat-GGML",  
    filename="llama-2-7b-chat.ggmlv3.q8_0.bin",  
    local_dir="."  
)
```

방법 3: CLI

```
huggingface-cli download TheBloke/Llama-2-7B-Chat-GGML llama-2-7b-chat.ggmlv3.q8_0.bin
```

5. CTransformers — 로컬에서 AI 실행

CTransformers란?

GGML/GGUF 모델을 Python에서 실행할 수 있게 해주는 라이브러리.
C/C++로 구현되어 CPU에서도 빠르게 동작.

```
pip install ctransformers      # CPU 버전
pip install ctransformers[cuda] # GPU 가속 버전
```

기본 사용법

```
from ctransformers import AutoModelForCausalLM

# 로컬 파일로 모델 불러오기
model = AutoModelForCausalLM.from_pretrained(
    './llama-2-7b-chat.ggmlv3.q8_0.bin',
    model_type='llama',
    config={
        'max_new_tokens': 512, # 최대 생성할 단어 수
        'temperature': 0.7     # 창의성 수치
    }
)

result = model("이메일을 작성해주세요.")
print(result)
```

LangChain과 함께 사용

```
from langchain_community.llms.ctransformers import CTransformers

llm = CTransformers(
    model='./llama-2-7b-chat.ggmlv3.q8_0.bin',
    model_type='llama',
    config={'max_new_tokens': 512, 'temperature': 0.01}
)

response = llm.invoke("안녕하세요, 자기소개 부탁드립니다.")
```

CTransformers의 한계

- 모델 파일을 직접 다운로드해서 경로 지정해야 함
 - 최신 모델로 업데이트가 번거로움
 - GPU 설정이 복잡함
- 이런 불편함을 해결한 것이 Ollama

6. GPU vs CPU 속도 실험

실험 환경: Google Colab (T4 GPU 무료 사용)

Google Colab이란?

구글이 제공하는 무료 클라우드 노트북 환경.

인터넷 브라우저에서 Python 코드 실행 가능. GPU도 무료로 사용 가능.

사용법: colab.research.google.com → 새 노트북 → 런타임 → T4 GPU 선택

실험 코드

```
from transformers import AutoModelForCausalLM
import time

prompt = "Hello, what is 2+2?"

# ① CPU 모드 (gpu_layers=0)
model_cpu = AutoModelForCausalLM.from_pretrained(
    "TheBloke/Llama-2-7B-Chat-GGUF",
    model_file="llama-2-7b-chat.Q4_K_M.gguf",
    model_type="llama",
    gpu_layers=0,      # 모든 계산을 CPU에서 처리
)
start = time.time()
out = model_cpu(prompt, max_new_tokens=64)
cpu_time = time.time() - start
print(f"CPU 시간: {cpu_time:.1f}초")

# ② GPU 모드 (gpu_layers=50)
model_gpu = AutoModelForCausalLM.from_pretrained(
    "TheBloke/Llama-2-7B-Chat-GGUF",
    model_file="llama-2-7b-chat.Q4_K_M.gguf",
    model_type="llama",
    gpu_layers=50,     # 50개 레이어를 GPU에서 처리
)
start = time.time()
out = model_gpu(prompt, max_new_tokens=64)
gpu_time = time.time() - start
print(f"GPU 시간: {gpu_time:.1f}초")
print(f"GPU가 {cpu_time/gpu_time:.1f}배 빠름")
```

gpu_layers란?

AI 모델은 여러 층(레이어)으로 구성됨.

```
gpu_layers=0 → 전부 CPU (느림)
gpu_layers=20 → 20개 레이어만 GPU (중간)
gpu_layers=50 → 50개 레이어를 GPU (빠름)
gpu_layers=-1 → 가능한 전부 GPU (최대 속도)
```

실험 결과 (T4 GPU 기준)

모드	처리 시간
CPU만	~60초
GPU (50 레이어)	~5초
속도 차이	약 12배

결론: 일반 PC(CPU만 있을 때)는 응답 1개에 1분 이상 걸림.
GPU가 없다면 Ollama처럼 최적화된 도구를 사용하거나,
Groq처럼 클라우드 GPU를 빌리는 것이 현실적.

7. Ollama — 더 쉬운 로컬 AI 실행

Ollama란?

로컬 LLM을 Docker처럼 명령어 하나로 관리하는 도구.

```
# 설치: https://ollama.com 에서 다운로드 후 설치

# 모델 다운로드 (자동으로 최적화된 버전 선택)
ollama pull llama3.1:8b

# 모델과 직접 대화 (터미널)
ollama run llama3.1:8b

# 설치된 모델 목록
ollama list

# 모델 삭제
ollama rm llama3:latest

# 서버 실행 (앱에서 API로 사용할 때)
ollama serve
```

CTransformers vs Ollama 비교

항목	CTransformers	Ollama
모델 다운로드	HuggingFace에서 직접	ollama pull 모델명
경로 지정	필요 (파일 경로 직접 입력)	불필요 (모델명만)
업데이트	파일 교체	ollama pull 재실행
GPU 설정	복잡	자동
Python 연동	CTransformers 패키지	langchain-ollama 패키지
추천 대상	고급 사용자	초보자, 빠른 프로토타입

Python에서 Ollama 사용

```
from langchain_ollama.llms import OllamaLLM

# 모델 이름만 쓰면 끝!
llm = OllamaLLM(model="llama3.1:8b", temperature=0.7)

response = llm.invoke("이메일을 작성해주세요.")
print(response)
```

Ollama가 백그라운드에서 서버로 실행 중이어야 함 (ollama serve)

8. Streamlit — Python으로 웹 앱 만들기

Streamlit이란?

Python 코드만으로 웹 앱을 만드는 프레임워크.
HTML, CSS, JavaScript를 전혀 몰라도 됨.

```
pip install streamlit
streamlit run app.py # 실행하면 자동으로 브라우저 열림
```

자주 쓰는 Streamlit 요소


```

import streamlit as st

# — 페이지 기본 설정 —————
st.set_page_config(
    page_title="앱 이름",          # 탭 제목
    page_icon="",                 # 탭 아이콘
    layout="centered",            # 'centered' 또는 'wide'
    initial_sidebar_state="collapsed" # 사이드바 숨김
)

# — 텍스트 출력 —————
st.header("큰 제목")
st.subheader("중간 제목")
st.write("일반 텍스트 또는 변수")
st.markdown("***굵게**", *기울게*, [링크](url))
st.code("코드 블록", language=None) # 우상단에 복사 버튼 자동 생성

# — 입력 받기 —————
선택값 = st.selectbox("라벨", ["옵션1", "옵션2"])
긴글 = st.text_area("라벨", height=100)
짧은글 = st.text_input("라벨")
클릭여부 = st.button("버튼 텍스트")

# — 레이아웃 —————
col1, col2 = st.columns(2)      # 화면을 2열로 나눔
with col1:
    st.write("왼쪽 내용")
with col2:
    st.write("오른쪽 내용")

with st.expander("펼쳐보기"):    # 접고 펼칠 수 있는 영역
    st.write("숨겨진 내용")

# — 상태 표시 —————
with st.spinner("처리 중..."):   # 로딩 애니메이션
    결과 = 오래_걸리는_함수()

st.success("성공 메시지")
st.warning("경고 메시지")
st.error("오류 메시지")

```

Streamlit 작동 원리

사용자가 버튼 클릭 또는 값 변경
↓
Python 스크립트 전체가
위에서 아래로 다시 실행됨
↓
변경된 부분만 화면에 업데이트

이 특성 때문에:

```
# 버튼을 클릭했을 때만 실행되도록 조건문 사용
submit = st.button("생성하기")
if submit:
    결과 = AI호출()
    st.write(결과)
```

session_state — 새로고침해도 데이터 유지

Streamlit은 버튼 클릭마다 스크립트 전체가 재실행됨.
'session_state'를 쓰면 데이터를 세션 동안 유지할 수 있음.

```
# 초기화 (처음 한 번만 실행됨)
if "history" not in st.session_state:
    st.session_state.history = []

# 데이터 추가
st.session_state.history.append("새 이메일 내용")

# 데이터 읽기
for item in st.session_state.history:
    st.write(item)
```

9. LangChain — AI 연동 표준화

LangChain이란?

다양한 LLM(AI 모델)과의 연동을 표준화한 라이브러리.
모델을 바꿔도 코드를 거의 수정하지 않아도 됨.

```
pip install langchain-core langchain-community langchain-ollama langchain-groq
```

LangChain 패키지 구조

패키지	역할
-----	----

langchain-core	핵심 기능 (PromptTemplate, OutputParser 등)
langchain-community	서드파티 연동 (CTransformers 등)
langchain-ollama	Ollama 전용
langchain-groq	Groq 전용
langchain-google-genai	Google Gemini 전용

PromptTemplate — 빈칸 채우기 질문 양식

```
from langchain_core.prompts import PromptTemplate

# {중괄호}로 빈칸 만들기
template = """
{topic} 주제로 이메일을 {tone}로 작성해주세요.
보낸 사람: {sender}
받는 사람: {recipient}
"""

prompt = PromptTemplate(
    input_variables=["topic", "tone", "sender", "recipient"],
    template=template
)

# 빈칸 채우기
완성된_질문 = prompt.format(
    topic="회의 일정",
    tone="격식체",
    sender="김철수",
    recipient="이영희"
)

# 결과: "회의 일정 주제로 이메일을 격식체로 작성해주세요. 보낸 사람: 김철수..."
```

방법 1: invoke 방식 (이 프로젝트 사용)

```
from langchain_ollama.llms import OllamaLLM

llm = OllamaLLM(model="llama3.1:8b", temperature=0.7)
response = llm.invoke(완성된_질문)
```

방법 2: 체인(Chain) 방식 (현대적 방법)

```

from langchain_core.output_parsers import StrOutputParser

# 파이프()로 연결: 프롬프트 → AI → 텍스트 변환
chain = prompt | llm | StrOutputParser()

response = chain.invoke({
    "topic": "회의 일정",
    "tone": "격식체",
    "sender": "김철수",
    "recipient": "이영희"
})

```

체인 방식의 장점: 중간에 단계를 추가하거나 교체하기 쉬움

LangChain 덕분에 가능한 것: 모델 교체가 쉬움

```

# Ollama (로컬)
from langchain_ollama.llms import OllamaLLM
llm = OllamaLLM(model="llama3.1:8b")

# Groq (클라우드)
from langchain_groq import ChatGroq
llm = ChatGroq(model="llama-3.1-8b-instant")

# Google Gemini (클라우드)
from langchain.chat_models import init_chat_model
llm = init_chat_model(model="gemini-flash-latest", model_provider="google_genai")

# → llm 변수만 바꾸면 나머지 코드는 동일!

```

10. Groq — 무료 클라우드 AI API

Groq란?

LPU(Language Processing Unit)라는 전용 칩으로 LLM을 매우 빠르게 실행하는 클라우드 서비스.

- 웹사이트: <https://console.groq.com>
- 무료 (일일 사용량 제한 있음)
- Llama, Mistral 등 오픈소스 모델 지원
- 로컬 Ollama보다 훨씬 빠름

API 키 발급 방법

1. <https://console.groq.com> 접속
2. Sign Up → 계정 생성
3. API Keys 메뉴 → Create API Key
4. 키 복사 (gsk_로 시작)

주요 Groq 모델

모델	특징
llama-3.1-8b-instant	빠름, 이 프로젝트 사용
llama-3.3-70b-versatile	더 높은 품질
mixtral-8x7b-32768	긴 컨텍스트 지원

Python에서 사용

```
from langchain_groq import ChatGroq
from langchain_core.output_parsers import StrOutputParser

llm = ChatGroq(
    model="llama-3.1-8b-instant",
    temperature=0.7,
    api_key="gsk_XXXXXXXXXXXX" # 환경 변수로 관리할 것!
)

chain = prompt | llm | StrOutputParser()
response = chain.invoke({...})
```

Ollama vs Groq 비교

항목	Ollama (로컬)	Groq (클라우드)
비용	완전 무료	무료 (사용량 제한)
속도	PC 사양 의존	매우 빠름
인터넷	불필요	필요
개인정보	완벽 보호	서버로 전송
설치	Ollama 설치 필요	API 키만 필요
웹 배포	불가	가능

11. 보안 — API 키 안전하게 관리하기

API 키란?

클라우드 서비스(Groq, OpenAI 등)를 사용할 때 신원을 증명하는 비밀번호.
유출되면 다른 사람이 내 계정으로 무단 사용 가능 → 절대 공개 금지.

위험한 방법 (하면 안 됨!)

```
# 절대 이렇게 하면 안 됨 — GitHub에 올리면 전 세계에 노출
api_key = "gsk_XXXXXXXXXXXXXXXXXXXX"
```

안전한 방법 — .env 파일 사용

1단계: .env 파일 만들기

```
# 프로젝트 폴더에 .env 파일 생성
GROQ_API_KEY=gsk_XXXXXXXXXXXX
```

2단계: Python에서 읽기

```
pip install python-dotenv

from dotenv import load_dotenv
import os

load_dotenv() # .env 파일 로드
api_key = os.getenv("GROQ_API_KEY") # 환경 변수 읽기
```

3단계: .gitignore에 등록 (가장 중요!)

```
# .gitignore 파일
.env      ← 이 줄이 있으면 .env는 git에 절대 포함 안 됨
```

.env.example — 팀원과 안전하게 공유

```
# .env.example (실제 키 없이 어떤 키가 필요한지만 표시)
GROQ_API_KEY=your_groq_api_key_here
```

팀원이 받아서 할 일:

1. .env.example 파일을 복사해서 .env로 이름 변경
2. 각자 자신의 API 키를 입력

.gitignore 파일이란?

git이 추적하지 않을 파일 목록을 지정하는 파일.

```
# API 키 — 절대 올리면 안 됨
.env

# 모델 파일 — 용량이 너무 큼 (7GB)
*.bin
*.gguf

# Python 자동 생성 파일 — 불필요
__pycache__/_
*.pyc
```

12. Git과 GitHub — 코드 버전 관리

Git이란?

코드의 변경 이력을 관리하는 도구.
"언제, 누가, 무엇을 수정했는지" 기록.

GitHub란?

Git 저장소를 인터넷에 올려 공유하는 플랫폼.

기본 명령어

```
# 저장소 초기화 (처음 한 번)
git init

# 변경된 파일 스테이징 (올릴 파일 선택)
git add app.py requirements.txt .gitignore
# .env는 .gitignore에 있으므로 자동 제외됨

# 커밋 (이력 저장)
git commit -m "기능 추가: 말투 선택 기능"

# GitHub에 올리기
git push origin main

# 현재 상태 확인
git status

# 변경 이력 보기
git log
```

GitHub CLI (gh) — 명령어로 GitHub 작업

```
# 로그인 상태 확인
gh auth status

# 저장소 생성 + 코드 올리기 한 번에
gh repo create email-generator ₩
--public ₩
--description "AI 이메일 생성기" ₩
--source=. ₩
--remote=origin ₩
--push
```

13. Streamlit Cloud — 웹 배포

Streamlit Cloud란?

Streamlit 앱을 무료로 인터넷에 배포해주는 서비스.
GitHub 저장소만 연결하면 자동으로 앱이 배포됨.

배포 순서

- ① GitHub에 코드 올리기 (git push)
- ↓
- ② share.streamlit.io 접속 → GitHub 로그인
- ↓
- ③ "Create app" → 저장소, 브랜치, 파일 선택
- ↓
- ④ Secrets에 API 키 입력 (.env 역할)
- ↓
- ⑤ "Deploy!" 클릭 → 1~2분 후 완료

Streamlit Cloud Secrets 설정

Streamlit Cloud는 `.env` 대신 Secrets 기능으로 API 키 관리.

```
# Streamlit Cloud → 앱 설정 → Secrets 탭에 입력
GROQ_API_KEY = "gsk_xxxxxxxxxxxx"
```

Python 코드에서는 동일하게 `os.getenv()`로 읽힘:

```
api_key = os.getenv("GROQ_API_KEY") # 로컬: .env / 클라우드: Secrets
```

이 프로젝트 배포 결과

GitHub 저장소: <https://github.com/artistjini/email-generator>
배포된 앱 URL: <https://jini-email-generator-01.streamlit.app>

14. 전체 프로젝트 코드 해설

[app_groq.py 전체 흐름](#)

```

import os
import streamlit as st
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_groq import ChatGroq

# ① .env에서 API 키 로드
load_dotenv()

# ② 대화 기록 초기화 (새로고침해도 유지)
if "history" not in st.session_state:
    st.session_state.history = []

def getLLMResponse(form_input, email_sender, email_recipient, language, tone):

    # ③ AI 모델 연결
    llm = ChatGroq(
        model="llama-3.1-8b-instant",
        temperature=0.7,
        api_key=os.getenv("GROQ_API_KEY"),
    )

    # ④ 언어·말투에 따라 질문 틀 선택
    if language == "한국어":
        tone_text = "격식체(존댓말)" if tone == "격식체" else "비격식체(친근한 말투)"
        template = """
{email_topic} 주제를 {tone}로 이메일을 작성해주세요.
보낸 사람: {sender} / 받는 사람: {recipient}
언어: {language}. 한문 제외.
이메일 내용:
"""
    else:
        tone_text = "formal" if tone == "격식체" else "informal and friendly"
        template = """
Write a {tone} email about {email_topic}.
Sender: {sender} / Recipient: {recipient}
Language: {language}
Email content:
"""

    # ⑤ 프롬프트 템플릿 생성
    prompt = PromptTemplate(
        input_variables=["email_topic", "sender", "recipient", "language", "tone"],
        template=template,
    )

    # ⑥ 체인 구성 및 실행: 프롬프트 → AI → 텍스트
    chain = prompt | llm | StrOutputParser()
    response = chain.invoke({

```

```

        "email_topic": form_input,
        "sender": email_sender,
        "recipient": email_recipient,
        "language": language,
        "tone": tone_text,
    })

    return response

# ⑦ 페이지 설정
st.set_page_config(page_title="이메일 생성기 ", page_icon="", layout="centered")
st.header("이메일 생성기 ")

# ⑧ API 키 없으면 앱 중단
if not os.getenv("GROQ_API_KEY"):
    st.error(" GROQ_API_KEY가 없습니다. .env 파일을 확인하세요.")
    st.stop()

# ⑨ 사용자 입력 UI
language_choice = st.selectbox("언어:", ["한국어", "English"])
tone_choice = st.selectbox("말투:", ["격식체", "비격식체"])
form_input = st.text_area("이메일 주제", height=100)

col1, col2 = st.columns(2)
with col1:
    email_sender = st.text_input("보낸 사람")
with col2:
    email_recipient = st.text_input("받는 사람")

# ⑩ 버튼 클릭 시 생성
if st.button("생성하기", type="primary"):
    if not form_input or not email_sender or not email_recipient:
        st.warning("모든 항목을 입력해주세요.")
    else:
        with st.spinner("생성 중..."):
            response = getLLMResponse(
                form_input, email_sender, email_recipient, language_choice, tone_choice
            )
            st.session_state.history.append({
                "주제": form_input, "이메일": response,
                "말투": tone_choice, "언어": language_choice
            })

# ⑪ 최신 결과 표시
if st.session_state.history:
    latest = st.session_state.history[-1]
    st.divider()
    st.subheader("생성된 이메일")
    st.markdown(latest["이메일"])
    st.caption("복사하려면:")

```

```
st.code(latest["이메일"], language=None) # 우상단 복사 버튼 자동 생성
```

```
# ⑫ 이전 기록 표시
if len(st.session_state.history) > 1:
    st.divider()
    st.subheader("이전 기록")
    for i, item in enumerate(reversed(st.session_state.history[:-1])):
        with st.expander(f"{{{item['주제']}}} {{{item['말투']}}} / {{{item['언어']}}}"):
            st.markdown(item["이메일"])
            st.code(item["이메일"], language=None)
```

15. 핵심 개념 총정리

용어 사전

용어	설명
LLM	대형 언어 모델. 텍스트를 이해하고 생성하는 AI
Llama	Meta가 무료 공개한 오픈소스 LLM 시리즈
파라미터(B)	모델 크기 단위. 7B=70억 개 가중치
GGML/GGUF	CPU에서 LLM 실행을 위한 경량 파일 포맷
양자화	모델을 압축하는 기술 (Q8>Q4>Q2, 숫자 낮을수록 작고 품질 낮음)
HuggingFace	AI 모델 공유 플랫폼 (AI계의 GitHub)
CTransformers	GGML 모델을 Python에서 실행하는 라이브러리
Ollama	로컬 LLM을 쉽게 설치·관리하는 도구
Groq	무료 클라우드 LLM API (매우 빠름)
Streamlit	Python으로 웹 앱을 만드는 프레임워크
LangChain	다양한 LLM 연동을 표준화한 라이브러리
PromptTemplate	AI 질문의 빈칸 채우기 양식
temperature	AI 창의성 수치 (0=일정, 1=창의적)
session_state	Streamlit에서 새로고침해도 데이터 유지하는 방법
API 키	클라우드 서비스 사용 시 신원 증명 비밀번호
.env	API 키 등 민감 정보를 코드와 분리해 저장하는 파일
.gitignore	git이 추적하지 않을 파일 목록
Git	코드 변경 이력 관리 도구
GitHub	Git 저장소를 인터넷에 공유하는 플랫폼

이 프로젝트에서 사용한 기술 조합

[로컬 버전]	[웹 배포 버전]
Streamlit (UI)	Streamlit (UI)
↓	↓
LangChain (연결)	LangChain (연결)
↓	↓
Ollama (실행)	Groq API (실행)
↓	↓
llama3.1:8b (모델)	llama-3.1-8b-instant (모델)
내 컴퓨터에서 실행	클라우드 서버에서 실행
인터넷 불필요	인터넷 필요
무료	무료 (사용량 제한)

배운 것을 활용하는 법

이 프로젝트에서 배운 기술로 만들 수 있는 것들:

- 블로그 글 생성기 (주제 입력 → 글 자동 생성)
- 번역기 (언어 선택 → 번역)
- 코드 리뷰어 (코드 붙여넣기 → 피드백)
- 면접 질문 생성기 (직무 입력 → 예상 질문 목록)
- 레시피 추천기 (재료 입력 → 요리법 생성)

→ Streamlit UI + LangChain + Groq(또는 Ollama) 조합은 모두 동일

학습 날짜: 2026-07-03

프로젝트: <https://github.com/artistjini/email-generator>

배포 앱: <https://jini-email-generator-01.streamlit.app>